

January 2026 CSE 406: Cyber Security Sessional

Online 1: Buffer Overflow

Section: C1

Time: **25 minutes**

Objective

You are given a vulnerable C program (`target.c`) that contains a buffer overflow vulnerability. Your task is to **inject shellcode** into the buffer and redirect execution to it, thereby spawning a shell.

In case you need to generate machine code from assembly, you can use the following site:
<https://defuse.ca/online-x86-assembler.htm#disassembly>

Environment Setup

Run the following commands to set up your environment:

```
sudo sysctl -w kernel.randomize_va_space=0

sudo ln -sf /bin/zsh /bin/sh
```

Student ID Configuration

Open `target.c` and replace `<LAST_3_DIGITS_OF_YOUR_ID>` in the `#define STUDENT_ID` line with the **last 3 digits** of your Student ID. Do not keep leading zeroes.

For example, if your Student ID is 2105085, set:

```
#define STUDENT_ID 85
```

Compilation

```
gcc -m32 -o target -z execstack -fno-stack-protector target.c

sudo chown root target
sudo chmod 4755 target

touch badfile

gcc -m32 -o target_dbg -g -z execstack -fno-stack-protector target.c
```

IMPORTANT: Environment mismatch

Recall that the stack addresses inside GDB and outside in the shell often do not match due to environment variable mismatches and absolute/relative path usage while running the executable. For a smooth completion of the task, make sure to handle these issues.

Task

Write a Python exploit script (`exploit.py`) that generates a `badfile` such that when the program is run, it spawns a root shell.

Restrictions

1. **Do NOT modify `target.c`** (except for setting your `STUDENT_ID`)
2. Your exploit must work with the specific buffer size determined by **your** Student ID
3. You may NOT hardcode buffer sizes or offsets — determine them through GDB analysis

Submission

1. Create a folder with your student ID as name (i.e., 2105185)
2. Put `target.c` and `exploit.py` inside it.
3. Zip the folder and upload it.